Section A: Concept Application

1. Explain how relational databases help maintain accuracy in expense records.

Ans. Reduces Data Duplication**:** User and category information is stored only once in separate tables, avoiding repeated and inconsistent data.

Maintains Data Integrity: Primary keys and foreign keys ensure that every expense is linked to a valid user and category, preventing invalid records.

Ensures Data Consistency: Changes made to user or category information are reflected wherever they are referenced, keeping the data accurate.

Supports Accurate Queries**:** Relationships between tables allow users to retrieve correct expense details using SQL joins.

2. Why are constraints important in personal finance data?

Ans. Constraints are important in personal finance data because they ensure the data is accurate, consistent, and reliable.

1. Maintain Data Integrity: Constraints prevent invalid or incorrect data from being entered into the database.
2. Prevent Duplicate Records: The UNIQUE constraint ensures that values such as email addresses are not duplicated.
3. Ensure Required Data: The NOT NULL constraint makes sure important fields like amount, expense_date, and category_id are never left empty.
4. Maintain Relationships: PRIMARY KEY and FOREIGN KEY constraints ensure that every expense is linked to a valid user and category.
5. Validate Data: CHECK constraints can prevent invalid values, such as negative expense amounts.
6. Improve Data Accuracy: Constraints help keep financial records correct and consistent, making reports and analysis more reliable.

3. How does GROUP BY help analyze spending patterns?

Ans. SELECT category_name, SUM(amount) AS total_expense
FROM expenses

```
INNER JOIN categories
ON expenses.category_id = categories.category_id
GROUP BY category_name;
```

4. Explain a scenario where rollback is required during expense entry.

Ans. START TRANSACTION;

```
INSERT INTO expenses (user_id, category_id, amount, expense_date)

VALUES (1, 2, 5000, '2026-06-29');

-- An error occurs here

ROLLBACK;
```

Suppose a user is entering a new expense of ₹5,000 under the Rent category. While saving the record, the system detects that the category_id is invalid or the database connection fails.

5. How do views help users track monthly expenses efficiently?

Ans. Simplifies Data Access: Users can view monthly expense reports without writing complex SQL queries.

Shows Monthly Spending: A view can display the total expenses for each month, making it easy to monitor spending. Improves Performance: Frequently used reports can be accessed quickly through a view.

Enhances Security: Users can access only the required data without direct access to the underlying tables.

Supports Better Financial Planning: Monthly summaries help users identify spending trends and manage their budgets effectively.

```
CREATE VIEW monthly_expenses AS

SELECT

   MONTH(expense_date) AS month,

   SUM(amount) AS total_expense

FROM expenses

GROUP BY MONTH(expense_date);
```

6. Why use triggers for automatic category or balance updates?

Ans. CREATE TRIGGER update_balance

AFTER INSERT ON expenses

FOR EACH ROW

UPDATE users

SET balance = balance - NEW.amount

WHERE user_id = NEW.user_id;

Automatic Updates: Triggers can automatically update the account balance or category totals whenever a new expense is added, updated, or deleted.

Maintains Data Consistency: They ensure that related tables remain synchronized without requiring manual updates. Reduces Human Errors: Since updates happen automatically, the chances of forgetting to update balances or totals are minimized.

Saves Time: Users and developers do not need to write extra SQL queries to update related data.

Improves Data Accuracy: Triggers ensure that financial records are always up to date after every transaction.

Section B: SQL Hands-On

1. DDL understanding

   Ans.

   1.preventing “ orphaned ”data

   2.ensuring data consistency(cascading action).

   3.reliable data joins.

   4.moving the logic to the database level.

   If foreign keys are removed from the database schema, the system loses its ability to enforce data integrity automatically. The primary risk that emerges is the creation of **Orphaned Records**.

   The Risk of Orphaned Records

   When the foreign key constraint on user_id in the expenses table is removed, the database no longer verifies if an expense belongs to a valid user.

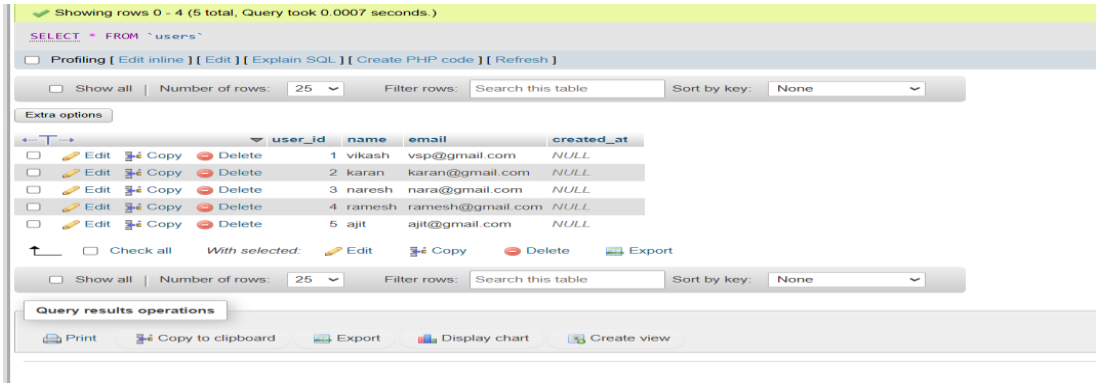
   If a user account is deleted from the users table, any expenses previously associated with that user_id will remain entirely untouched in the expenses table.

   This leads to several critical issues:

- **Inaccurate Financial Reporting:** Standard queries using INNER JOIN to calculate totals or display user expenses will completely omit these records because the corresponding user_id no longer exists in the users table.This results in missing financial data and distorted overall spending metrics.
- **Wasted Storage & Data Pollution:** The database accumulates "ghost" data—records that consume space but have no context, meaning, or owner.
- **Application Errors:** If the application attempts to fetch or display details for that specific expense, it will likely crash or throw errors when it tries to look up a user name or email that is missing from the system.

2. DML operations answers:
   1. insert 5 users name and emails

      ans.

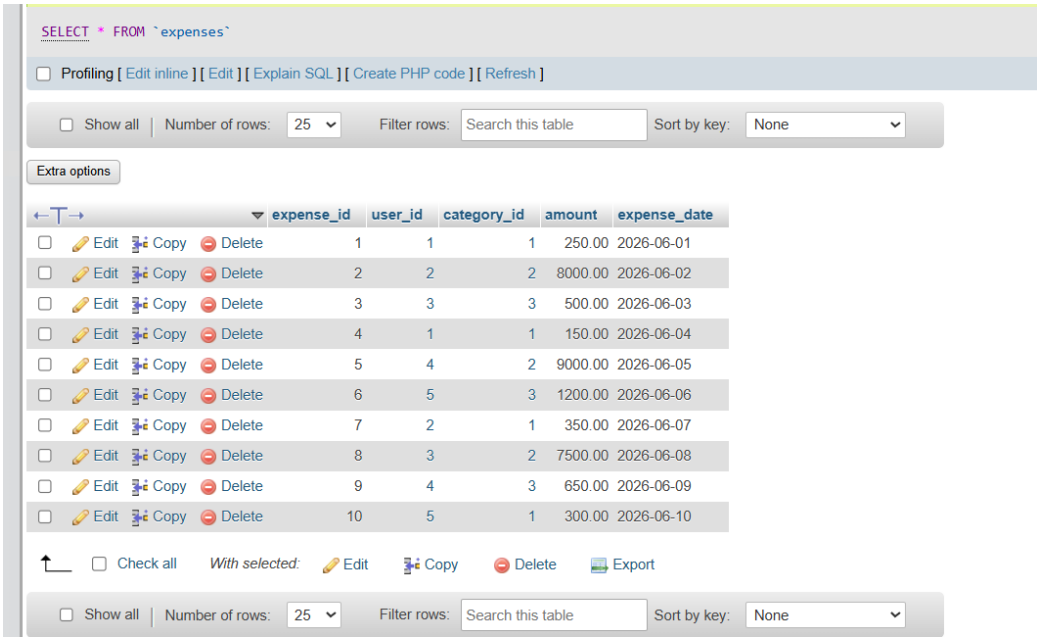


2. insert 3 categories.

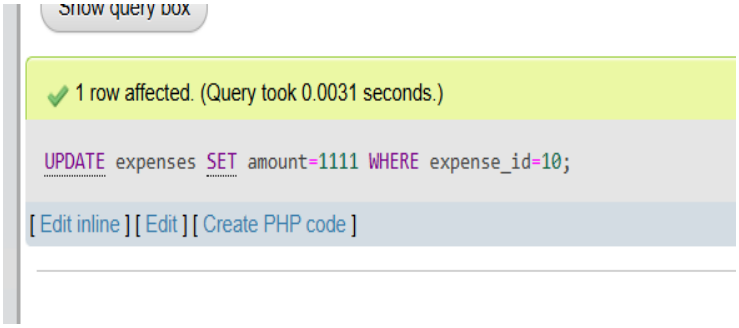
   Ans.

3. insert 10 expense record

   ans.

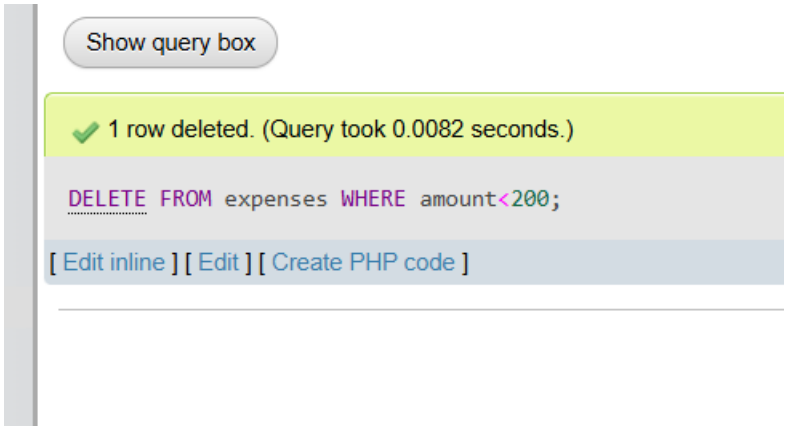


4. update query use



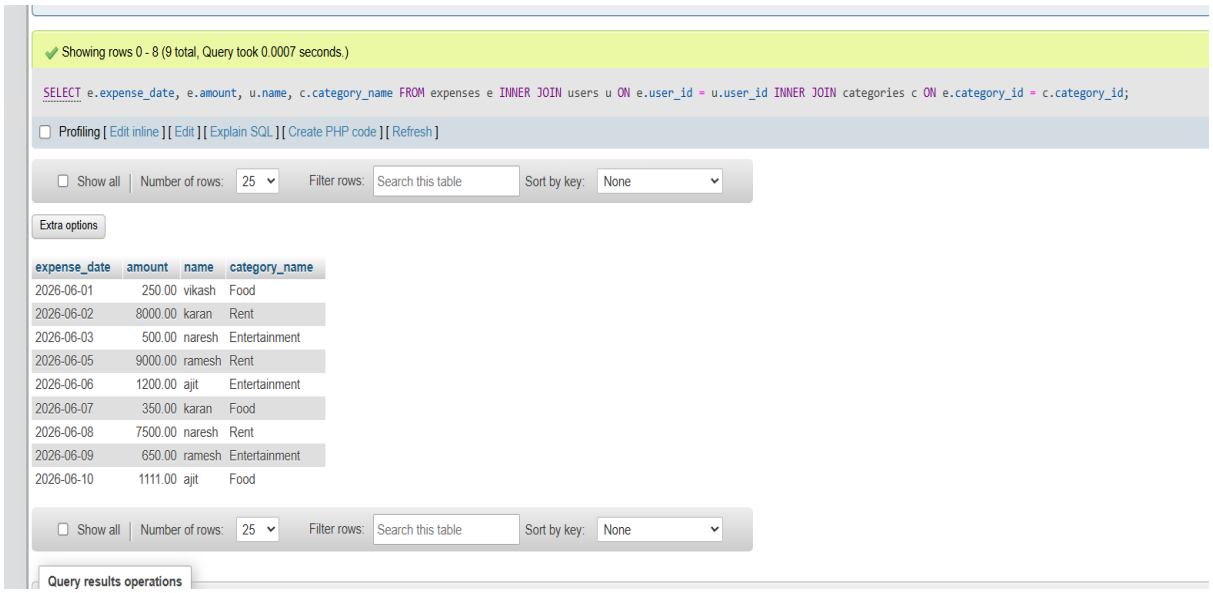
5. delete one expense

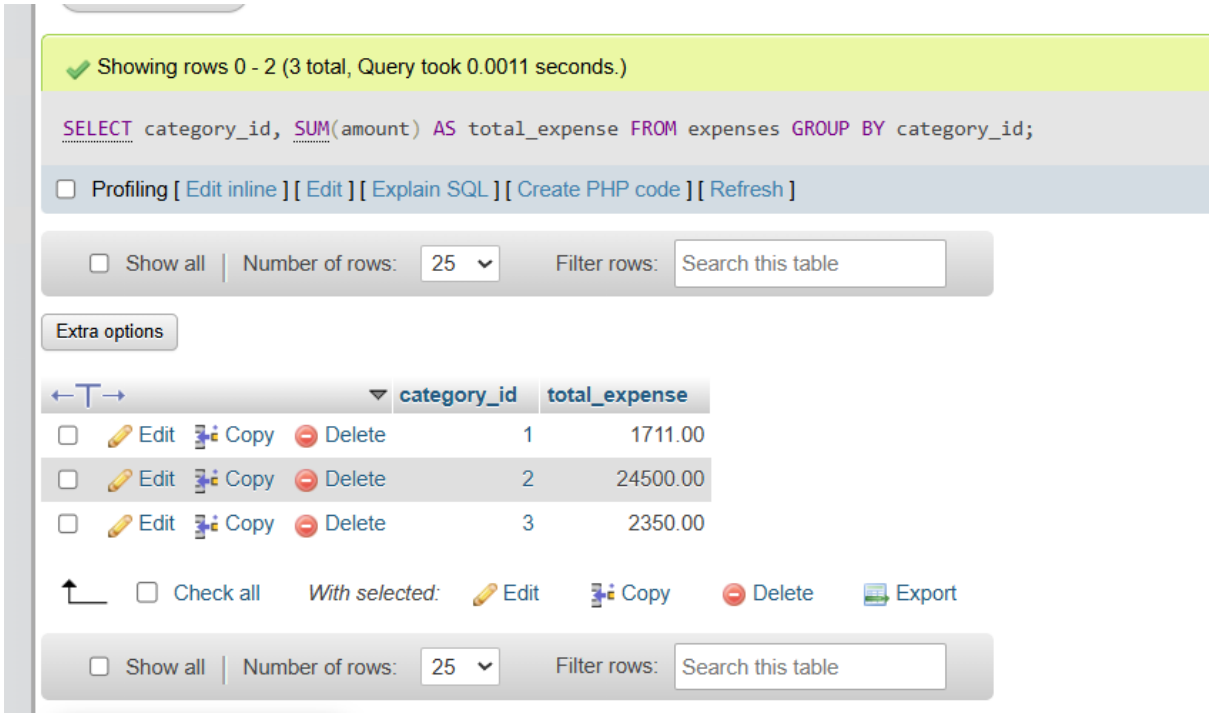
ans.



3. data retrivals.

Ans.



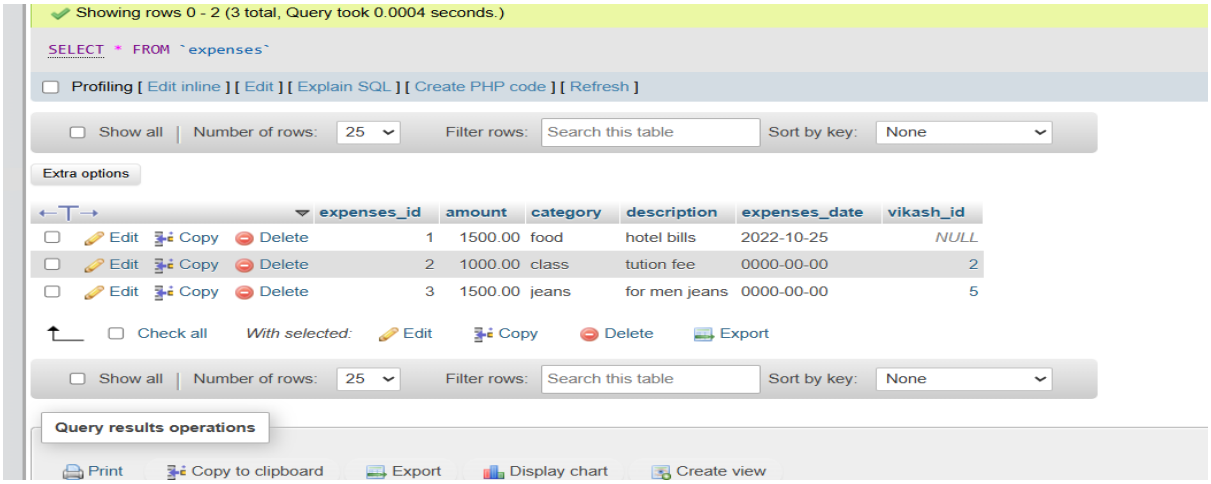

Showing rows 0 - 2 (3 total, Query took 0.0011 seconds.)
SELECT category_id, SUM(amount) AS total_expense FROM expenses GROUP BY category_id;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
category_id
total_expense
Edit Copy Delete 1 1711.00
Edit Copy Delete 2 24500.00
Edit Copy Delete 3 2350.00
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table

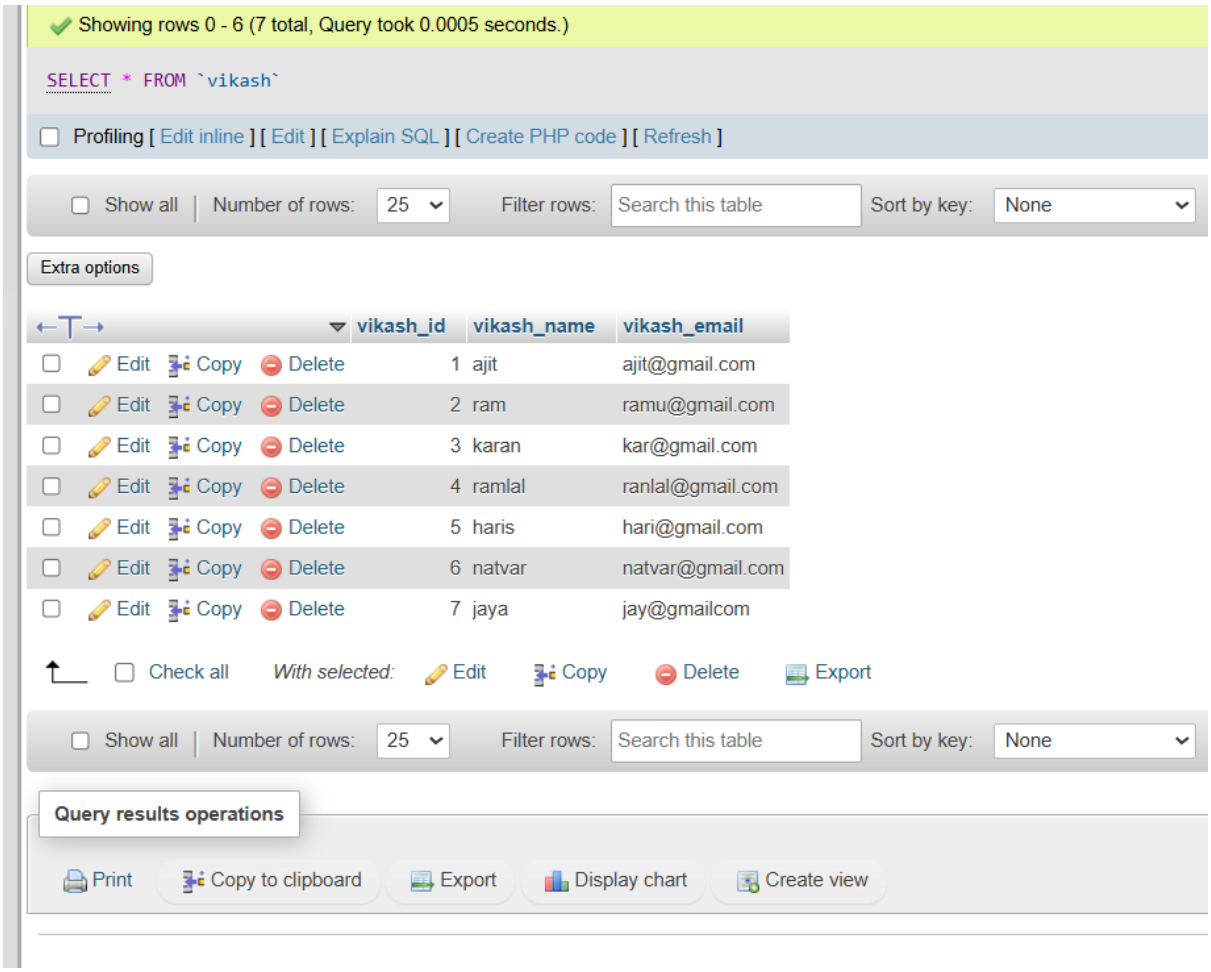
Section-c mini-Project

Crud program

1. insert data

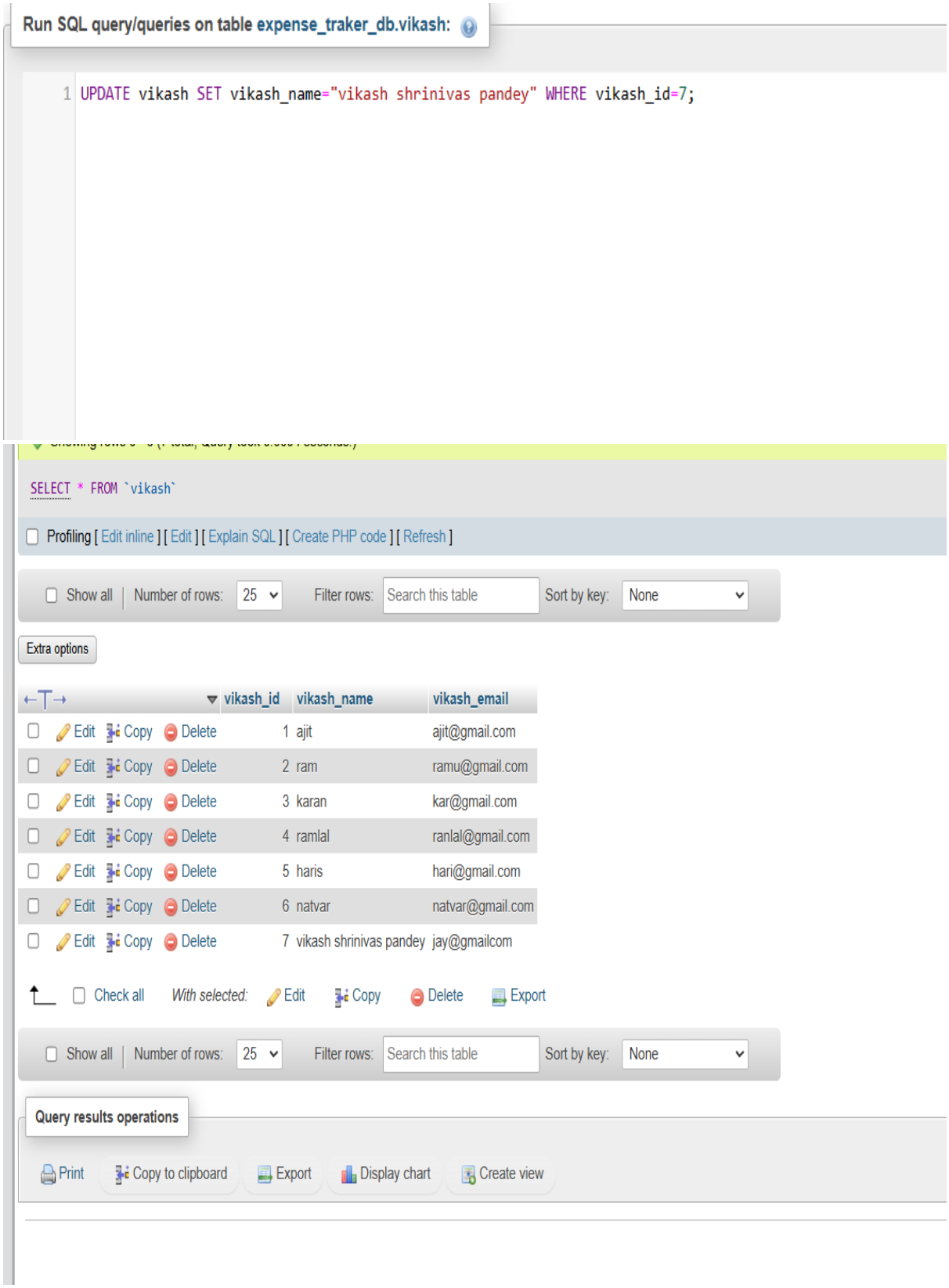
   ans.





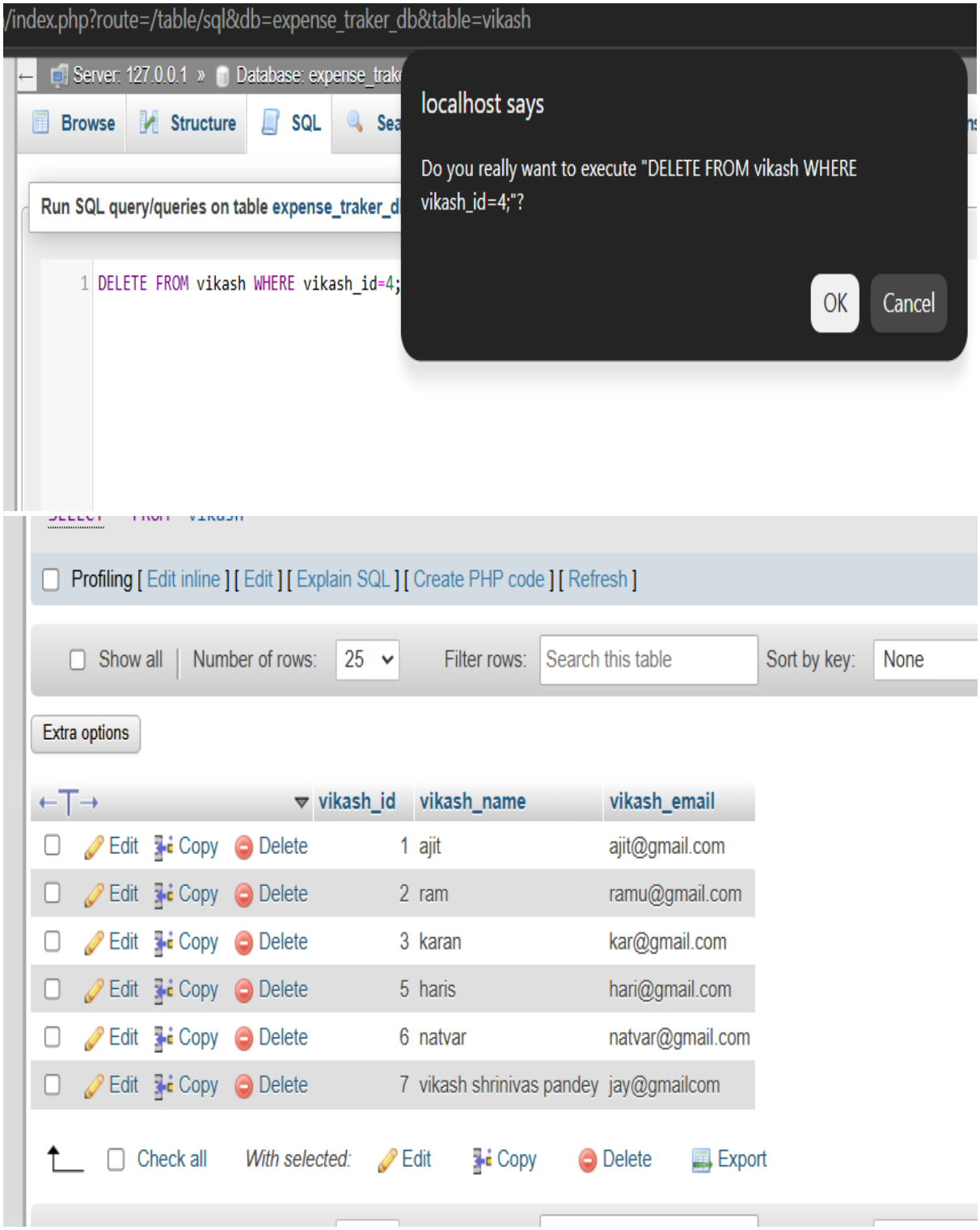
2. update data for table.

Ans.

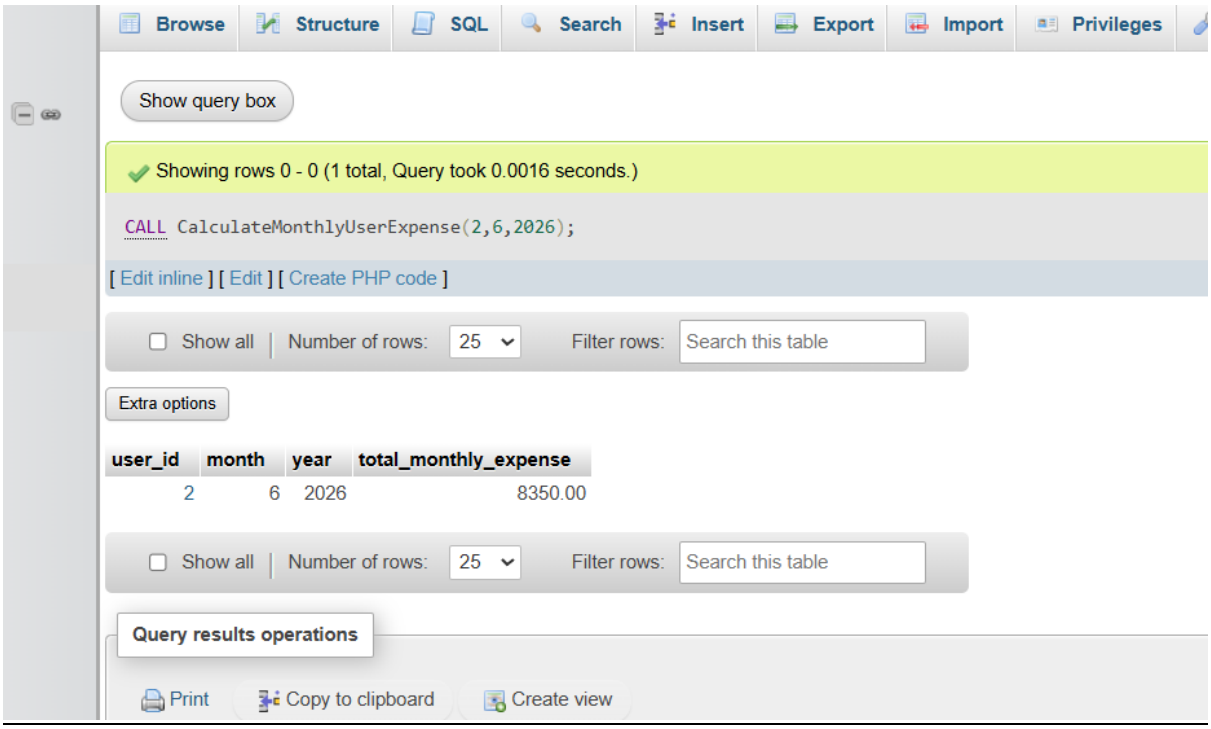


3. delete data for a table.

Ans.


/index.php?route=/table/sql&db=expense_traker_db&table=vikash
Server: 127.0.0.1 » Database: expense_trake
Browse
Structure
SQL
Run SQL query/queries on table expense_traker_d
1 DELETE FROM vikash WHERE vikash_id=4;
localhost says
Do you really want to execute "DELETE FROM vikash WHERE vikash_id=4;"?
OK
Cancel
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all
Number of rows: 25
Filter rows: Search this table
Sort by key: None
Extra options
vikash_id
vikash_name
vikash_email
Edit Copy Delete 1 ajit ajit@gmail.com
Edit Copy Delete 2 ram ramu@gmail.com
Edit Copy Delete 3 karan kar@gmail.com
Edit Copy Delete 5 haris hari@gmail.com
Edit Copy Delete 6 natvar natvar@gmail.com
Edit Copy Delete 7 vikash shrinivas pandey jay@gmailcom
Check all
With selected:
Edit
Copy
Delete
Export

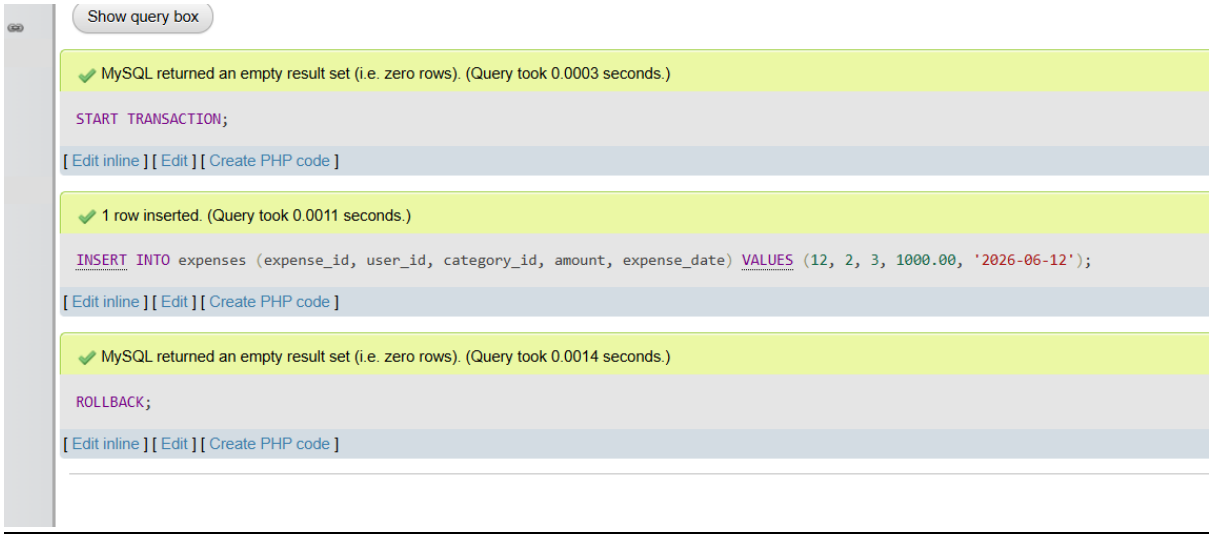
Write a store procedure to calculate monthly user expense:

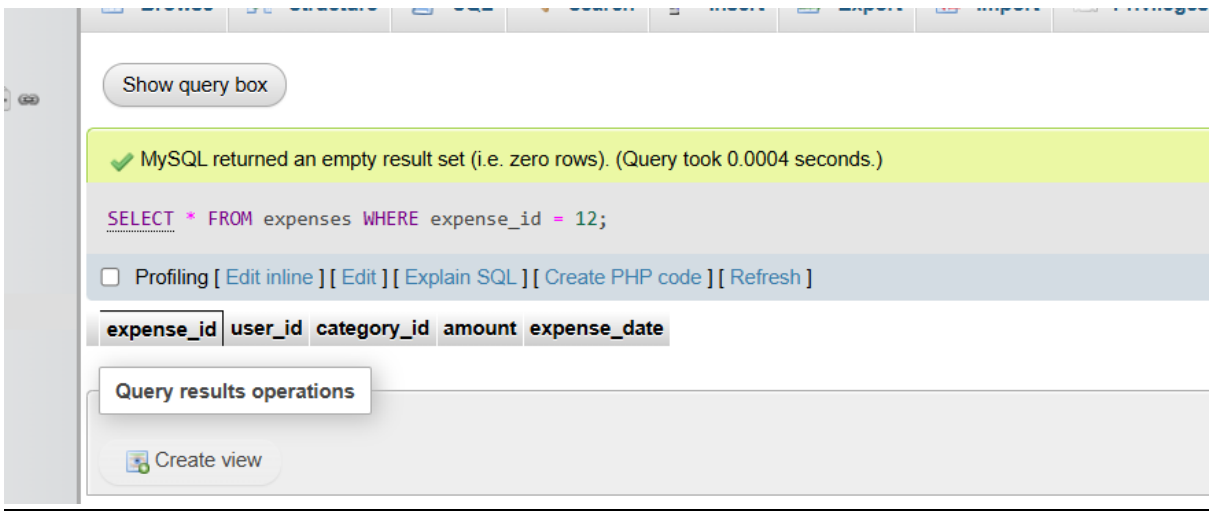


Commit and rollback


Show query box
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)
START TRANSACTION;
[Edit inline] [Edit] [Create PHP code]
1 row inserted. (Query took 0.0012 seconds.)
INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date) VALUES (11, 1, 2, 500.00, '2026-06-11');
[Edit inline] [Edit] [Create PHP code]
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0012 seconds.)
COMMIT;
[Edit inline] [Edit] [Create PHP code]


Show query box
Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)
SELECT * FROM expenses WHERE expense_id = 11;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
expense_id user_id category_id amount expense_date
Edit Copy Delete 11 1 2 500.00 2026-06-11
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print Copy to clipboard Export Display chart Create view


Show query box
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)
START TRANSACTION;
[Edit inline] [Edit] [Create PHP code]
1 row inserted. (Query took 0.0011 seconds.)
INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date) VALUES (12, 2, 3, 1000.00, '2026-06-12');
[Edit inline] [Edit] [Create PHP code]
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)
ROLLBACK;
[Edit inline] [Edit] [Create PHP code]


Show query box
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)
SELECT * FROM expenses WHERE expense_id = 12;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
expense_id
user_id
category_id
amount
expense_date
Query results operations
Create view